

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Question 1 – Regular Expression Pattern Generation

Problem Statement

For the given student details, formulate Regular Expression (Regex) patterns to validate the following fields:

- Email ID
- Mobile Number
- Strong Password
- Date of Birth (DD/MM/YYYY)
- Register Number

Write a Python program using the **re** module to validate each field.

Regex Patterns

Field	Regex Pattern
Email ID	<code>^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$</code>
Mobile Number	<code>^\+91-\d{10}\$</code>
Strong Password	<code>^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#\$%&!*])[A-Za-z\d@#\$%&!*]{8,}\$</code>
Date of Birth	<code>^\d{2}/\d{2}/\d{4}\$</code>
Register Number	<code>^\d{2}[A-Z]{4}\d{4}\$</code>

Pattern Description

Email ID

Matches a valid email address containing a username, @ symbol, domain name, and domain extension.

Mobile Number

Matches an Indian mobile number beginning with **+91-** followed by exactly **10 digits**.

Strong Password

Matches a password that:

- contains at least 8 characters,
- has at least one uppercase letter,

- has at least one lowercase letter,
- has at least one digit,
- has at least one special character (@ # \$ % & ! *).

Date of Birth

Matches dates in the format **DD/MM/YYYY**.

Register Number

Matches a register number consisting of:

- 2 digits (Year)
- 4 uppercase letters (Department Code)
- 4 digits (Roll Number)

Example: **23AIML1056**

Algorithm

1. Import the re module.
 2. Store the student details.
 3. Define the regex pattern for each field.
 4. Use re.match() to compare each value with its corresponding pattern.
 5. Display whether each value is **Valid** or **Invalid**.
-

Python Program

```
import re
```

```
student = {  
    "Email": "john.doe123@gmail.com",  
    "Mobile": "+91-9876543210",  
    "Password": "P@ssw0rd123",  
    "DOB": "15/08/2004",  
    "Register": "23AIML1056"  
}
```

```
patterns = {
```

```

>Email": r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$",
>Mobile": r"^\+91-\d{10}$",
>Password": r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!*])[A-Za-z\d@#%&!*]{8,}$",
>DOB": r"^\d{2}/\d{2}/\d{4}$",
>Register": r"^\d{2}[A-Z]{4}\d{4}$"
>
>}
```

```

for field, pattern in patterns.items():
    if re.match(pattern, student[field]):
        print(f'{field}: Valid')
    else:
        print(f'{field}: Invalid')
```

Output

```

Email: Valid
Mobile: Valid
Password: Valid
DOB: Valid
Register: Valid
```

Question 2 – Search for a String

Problem Statement

Write a Python program using the **re** module to search for the word **"AI"** in the given passage. Display the first occurrence, starting position, ending position, total number of occurrences, and print an appropriate message if the word is not found.

Regex Pattern

`\bAI\b`

Description: Matches the complete word **"AI"** using word boundaries.

Algorithm

1. Import the re module.
 2. Store the given passage in a string.
 3. Define the regex pattern `\bAI\b`.
 4. Use `re.search()` to find the first occurrence.
 5. Display the starting and ending positions using `start()` and `end()`.
 6. Use `re.findall()` to count all occurrences.
 7. If no match is found, display an appropriate message.
-

Python Program

```
import re
```

```
text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""
```

```
pattern = r"\bAI\b"
```

```
match = re.search(pattern, text)
```

```
if match:
```

```
    print("First occurrence found.")
    print("Starting Position:", match.start())
    print("Ending Position:", match.end())
    print("Total Occurrences:", len(re.findall(pattern, text)))
```

```
else:
```

```
    print("Word 'AI' not found.")
```

Output

First occurrence found.

Starting Position: 25

Ending Position: 27

Total Occurrences: 6

Question 3 – Split the Documentation into Sentences and Words

Problem Statement

Write a Python program using the **re.split()** method to perform the following operations on the given passage:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
 - Count the total number of sentences.
 - Split the passage into words using one or more whitespace characters as delimiters.
 - Count the total number of words.
 - Display the list of extracted sentences and words.
-

Regex Patterns

Sentence Split

[.!?]+

Word Split

\s+

Pattern Description

Sentence Split ([.!?]+)

- Splits the text wherever a period (.), question mark (?), or exclamation mark (!) appears.
- The + operator allows one or more punctuation marks to be treated as a single delimiter.

Word Split (\s+)

- Splits the text using one or more whitespace characters such as spaces, tabs, or new lines.
-

Algorithm

1. Import the re module.
 2. Store the given passage in a string.
 3. Use `re.split(r'[.!?]+' , text)` to split the passage into sentences.
 4. Remove empty sentences from the list.
 5. Use `re.split(r'\s+' , text)` to split the passage into words.
 6. Count the number of sentences and words.
 7. Display the extracted sentences, extracted words, and their counts.
-

Python Program

```
import re
```

```
text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""
```

```
# Split into sentences
```

```
sentences = re.split(r'[.!?]+' , text)
```

```
sentences = [s.strip() for s in sentences if s.strip()]
```

```
# Split into words
```

```
words = re.split(r'\s+' , text.strip())
```

```
print("Extracted Sentences:")
```

```
for s in sentences:
```

```
    print(s)
```

```
print("\nTotal Sentences:" , len(sentences))
```

```
print("\nExtracted Words:")
```

```
print(words)
```

```
print("\nTotal Words:", len(words))
```

Output

Extracted Sentences:

Artificial Intelligence (AI) is transforming industries across the world

AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences

Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making

As AI continues to evolve, professionals with AI skills are in high demand

Total Sentences: 4

Extracted Words:

```
['Artificial', 'Intelligence', '(AI)', 'is', 'transforming', ..., 'demand.']
```

Total Words: 54

Result

The given passage was successfully split into individual sentences and words using the `re.split()` method. The program displayed the extracted sentences, extracted words, and their respective counts, demonstrating text tokenization using Python Regular Expressions.

Question 4 – Email and Strong Password Validation

Problem Statement

A website requires users to register using an **Email ID** and a **Strong Password**. Write a Python program using the **re** module to validate the given Email ID and Strong Password based on the specified validation criteria.

Regex Patterns

Email ID

```
^[A-Za-z0-9]+[A-Za-z0-9._%+]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$
```

Strong Password

```
^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!*])\S{8,}$
```

Pattern Description

Email ID

- Begins with one or more letters or digits.
- May contain ., _, %, +, or - before @.
- Contains a valid domain name.
- Ends with a valid domain extension (.com, .org, .edu, or .in).

Strong Password

- Contains at least 8 characters.
- Contains at least one uppercase letter.
- Contains at least one lowercase letter.
- Contains at least one digit.
- Contains at least one special character (@, #, \$, %, &, !, *).
- Does not contain whitespace.

Algorithm

1. Import the re module.
2. Read the Email ID and Password.
3. Define the regex patterns for Email ID and Password.
4. Use re.fullmatch() to validate both inputs.
5. Display whether the Email ID and Password are valid or invalid.

Python Program

```
import re
```

```
email = "john.doe123@gmail.com"
```

```
password = "P@ssw0rd123"
```



```
email_pattern = r"^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$"
password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!*])\S{8,}$"
```

```
if re.fullmatch(email_pattern, email):
    print("Email ID: Valid")
```

```
else:
```

```
    print("Email ID: Invalid")
```

```
if re.fullmatch(password_pattern, password):
```

```
    print("Password: Valid")
```

```
else:
```

```
    print("Password: Invalid")
```

Output

Email ID: Valid

Password: Valid

Result

The program successfully validated the given Email ID and Strong Password using Python's `re` module. Both inputs satisfied the specified validation criteria and were identified as **Valid**.

Question 5 – Compile Once, Validate Many

Problem Statement

A company receives **1,000 Email IDs** from its customers and needs to validate each one efficiently. Instead of compiling the same Regular Expression repeatedly, write a Python program using the **`re.compile()`** method to compile the pattern once and reuse it to validate all Email IDs.

Regex Pattern

```
^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$
```

Pattern Description

- Begins with one or more letters or digits.
 - Allows ., _, %, +, and - before the @ symbol.
 - Contains a valid domain name.
 - Ends with one of the valid domain extensions: **.com**, **.org**, **.edu**, or **.in**.
 - The pattern is compiled once using `re.compile()` and reused for validating multiple Email IDs, improving performance.
-

Algorithm

1. Import the re module.
 2. Compile the Email ID regex pattern using `re.compile()`.
 3. Store the Email IDs in a list.
 4. Iterate through each Email ID.
 5. Use the compiled pattern to validate each Email ID.
 6. Display whether each Email ID is valid or invalid.
-

Python Program

```
import re

# Compile the regex pattern once
email_pattern = re.compile(
    r"^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$"
)

emails = [
    "john.doe123@gmail.com",
    "student@saveetha.edu",
    "user123@yahoo.in",
    "invalid@email",
    "@gmail.com"
]
```

for email in emails:

```
    if email_pattern.fullmatch(email):
```

```
        print(email, "-> Valid")
```

```
    else:
```

```
        print(email, "-> Invalid")
```

Output

john.doe123@gmail.com -> Valid

student@saveetha.edu -> Valid

user123@yahoo.in -> Valid

invalid@email -> Invalid

@gmail.com -> Invalid

Result

The Email ID validation pattern was compiled once using the **re.compile()** method and successfully reused to validate multiple Email IDs. This approach improves efficiency and performance when validating a large number of inputs.